VulnHub Machine: SkyTower: 1

Details	Information
Platform	**VulnHub**
Target Machine	**SkyTower: 1**
Target IP Address	**192.168.56.101**
Attacker Machine	**Kali Linux**
Attacker IP Address	**192.168.56.104**
Network Environment	**VirtualBox Host-Only Network**
Assessment Type	**Authorized Penetration Testing / CTF Lab**

FROM: Saniya Shaikh

Contents

1. Objective 3

2. Lab Environment 3

3. Target Information 3

4. Reconnaissance 3

Step 1 — Finding the Kali (Attacker) IP Address 4

Step 2 — Discovering Hosts on the Local Network 5

Step 3 — Scanning Candidate Hosts for Open Ports and Services 6

5. Web Enumeration 7

Step 4 — Viewing the Target Website in a Browser 7

Step 5 — Extracting the Login Form Details with curl 8

6. Exploitation — SQL Injection Authentication Bypass 9

Step 6 — Testing the Login Form for SQL Injection 9

Step 7 — Bypassing Authentication and Retrieving Leaked Credentials 10

7. Pivoting Through the Internal Proxy 11

Step 8 — Configuring Proxychains to Use the Squid Proxy 11

Step 9 — Connecting Over SSH Through the Proxy 13

8. Shell Stabilization 14

Step 10 — Bypassing the Forced Restrictive Shell 14

Step 11 — Removing the Malicious .bashrc 15

Step 12 — Confirming a Stable Shell 16

9. Local Enumeration 17

Step 13 — Checking sudo Privileges and Network Connections 17

Step 14 — Reviewing Web Application Source for Database Credentials 18

10. Credential Discovery — MySQL Database 18

Step 15 — Logging into MySQL with the Discovered Credentials 18

Step 16 — Selecting the SkyTech Database 19

Step 17 — Dumping the Login Table 20

11. Lateral Movement — Pivoting to the sara Account 21

Step 18 — Switching to sara Using the Discovered Credentials 21

12. Privilege Escalation — Sudo Wildcard Exploitation 22

Step 19 — Enumerating sara's Sudo Rights 22

Step 20 — Exploiting the Wildcard to Read Root's Files 23

13. Conclusion 24

1. Objective

The goal of this machine was to gain a foothold on the VulnHub machine "**SkyTower: 1**" and progressively escalate privileges until root-level (administrator-level) access was achieved. SkyTower is a boot2root machine themed around a fictional company, "SkyTech," and its retired employees' portal. Unlike CTF machines that hide numbered flags, SkyTower marks success through a final root-password disclosure left inside a file only the root account can reach.

This report documents every step actually performed during the assessment, in the order it happened, explaining what each command did, what it revealed, and why that result led to the next action taken.

2. Lab Environment

- Attacker machine: Kali Linux (user: saniya)
- Target machine: SkyTower: 1 (VulnHub boot2root VM)
- Network: isolated host-only/NAT network 192.168.56.0/24, interface eth0 on Kali
- Kali IP address: 192.168.56.104
- Target IP address: 192.168.56.103

3. Target Information

Item	Value
Attacker IP (Kali)	192.168.56.104
Target IP (SkyTower: 1)	192.168.56.103
Open ports	22/tcp (SSH, filtered from outside), 80/tcp (HTTP), 3128/tcp (HTTP-proxy)
Web server	Apache httpd 2.2.22 (Debian)
Proxy service	Squid http proxy 3.1.20
Web technology	PHP, MySQL back end
Operating System	Debian GNU/Linux (kernel 3.2.0-4-amd64)

4. Reconnaissance

Reconnaissance is the first stage of any penetration test. The goal here was to confirm the attacker's own network identity, discover which host on the local network was the target, and enumerate the services the target exposed.

Step 1 — Finding the Kali (Attacker) IP Address

Command: ip addr and ip route

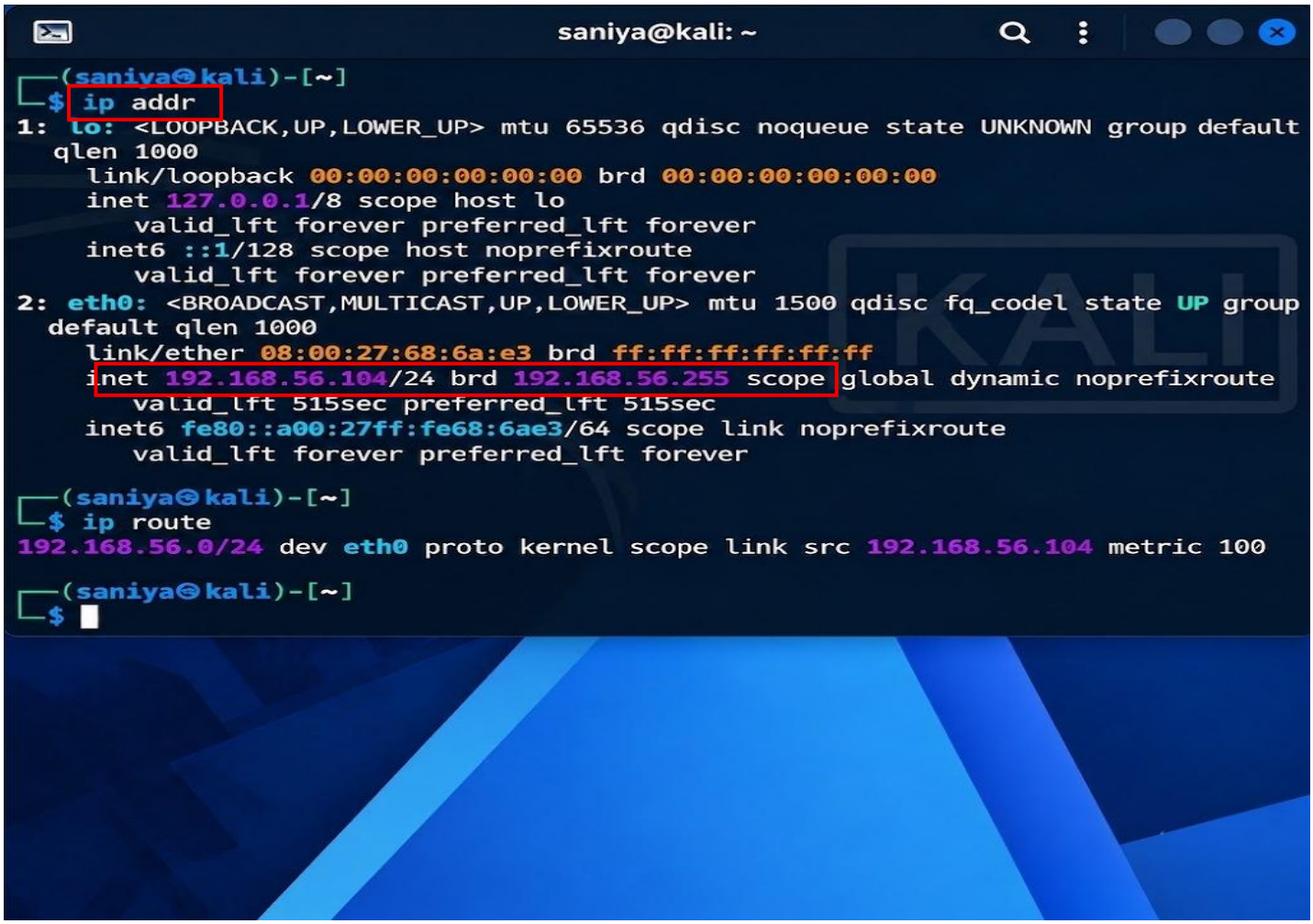


Figure 1: Kali Network Configuration

Description: The ip addr command lists all network interfaces on the Kali machine and their assigned IP addresses. The ip route command shows which network is reachable through which interface.

Result: Interface eth0 was assigned the IP address 192.168.56.104/24, and the route table confirmed that the 192.168.56.0/24 network is reachable through eth0.

Interpretation: 192.168.56.104 is the Kali (attacker) machine's own IP address on the lab network. Any target machine for this exercise must also sit on the 192.168.56.0/24 network.

Why this matters: Knowing the attacker's own IP is required later for confirming which side of any captured or proxied traffic belongs to Kali, and for verifying that scan results are not reflecting the attacker's own host.

Next step: With the attacker's network identified, the next task was to discover which other hosts existed on that network.

Step 2 — Discovering Hosts on the Local Network

Command: An ARP-based host discovery scan (netdiscover) was run against the 192.168.56.0/24 network.

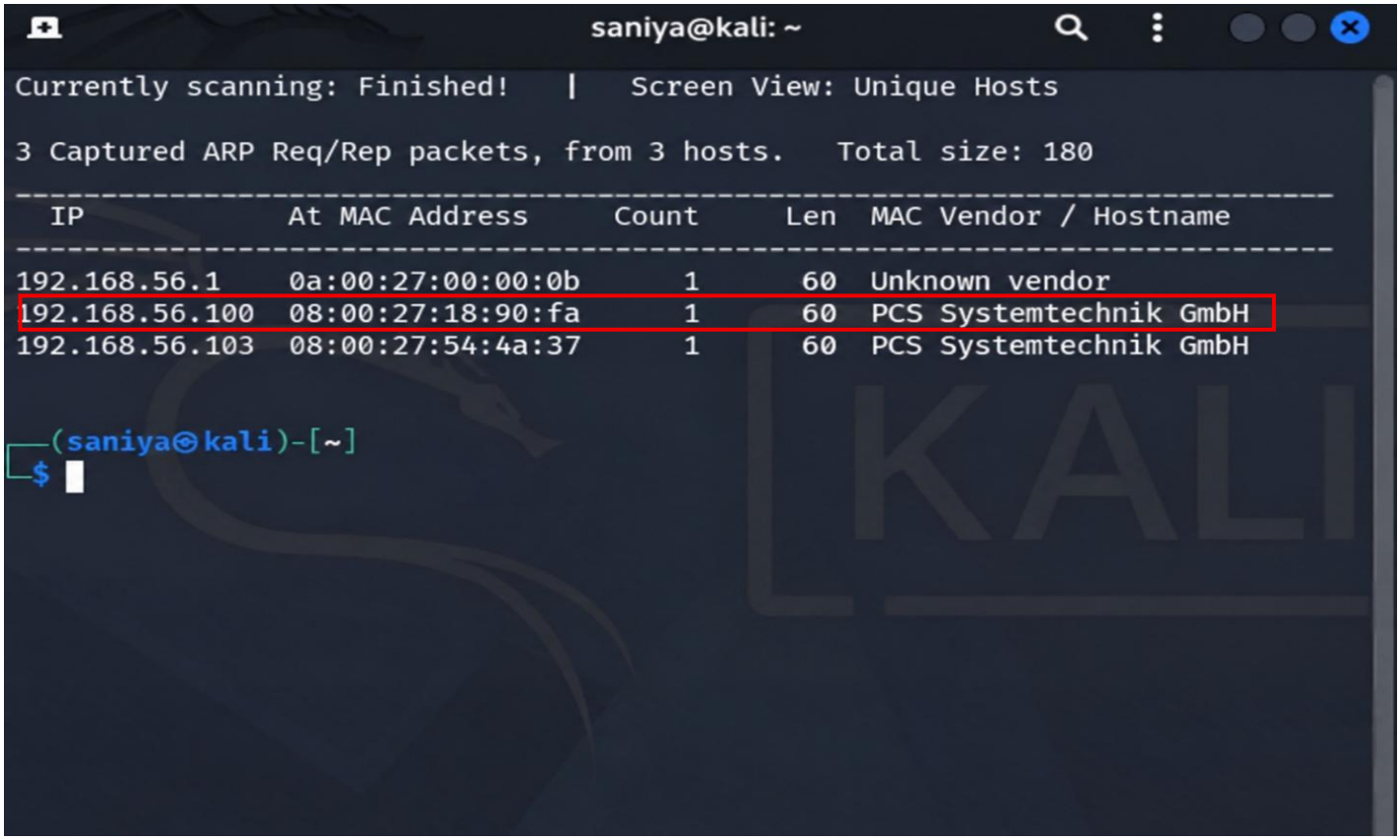


Figure 2: ARP Scan Results — Unique Hosts on the Local Network

Description: An ARP scan sends broadcast requests on the local network segment and records which hosts reply, along with their MAC address vendor. This does not rely on ICMP or TCP, so it works even against hosts that filter conventional pings or port scans.

Result: Three hosts replied: 192.168.56.1 (the VirtualBox host adapter), 192.168.56.100, and 192.168.56.103, the latter two both showing a "PCS Systemtechnik GmbH" MAC vendor, which is the vendor string VirtualBox assigns to guest VMs.

Interpretation: Both .100 and .103 are candidate VirtualBox guest machines and needed to be scanned individually to determine which one was the actual SkyTower target.

Why this matters: In an unfamiliar lab it is easy to scan the wrong host. Confirming candidates via ARP first avoids wasting time running heavier scans against irrelevant machines.

Next step: Both candidate hosts were scanned for open ports and running services.

Step 3 — Scanning Candidate Hosts for Open Ports and Services

Command: sudo nmap -sV -v 192.168.56.100 and sudo nmap -sV -v 192.168.56.103

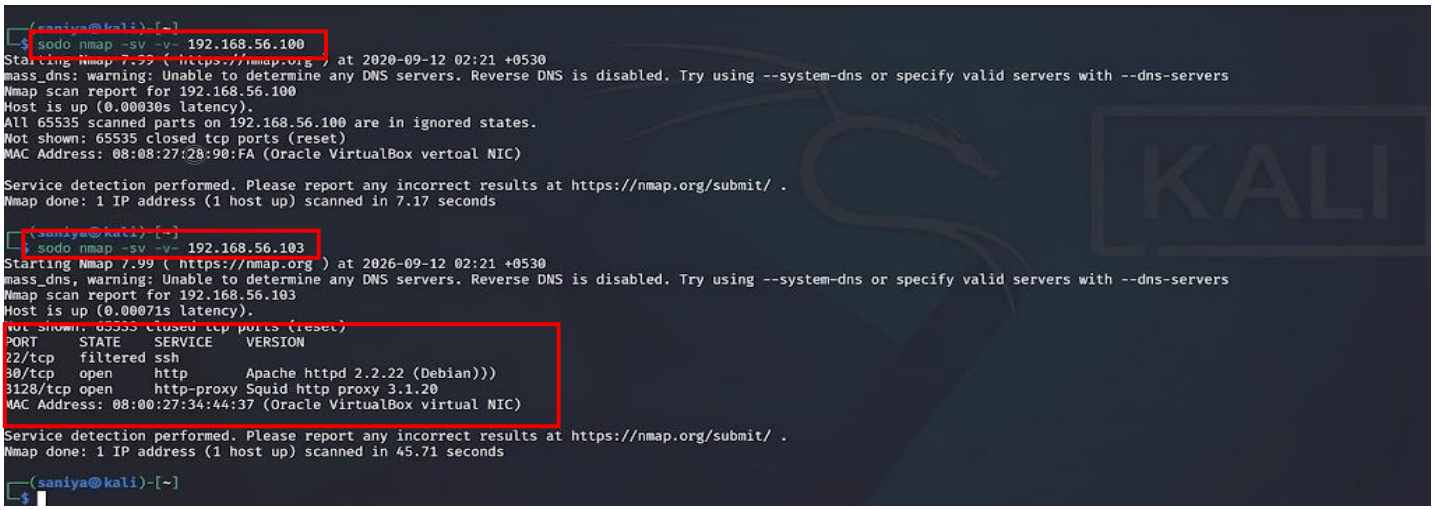


Figure 3: Nmap Service Scan Results for Both Candidate Hosts

Description: The -sV flag performs service and version detection on any open ports, and -v increases verbosity. Both discovered hosts were scanned in turn to identify which one was the SkyTower machine.

Result: 192.168.56.100 returned no open ports (all 65535 ports closed). 192.168.56.103 showed port 22 (SSH) filtered, port 80 (HTTP) open running Apache httpd 2.2.22 (Debian), and port 3128 open running an http-proxy identified as Squid 3.1.20.

Interpretation: 192.168.56.103 is confirmed as the actual SkyTower target. SSH being filtered (rather than closed or open) suggested it might only be reachable from specific sources — in this case, from the target itself through its own proxy, rather than from the outside. Port 3128 running Squid was an unusual and significant finding, since internal proxies are sometimes used to reach services that are not directly exposed.

Why this matters: This told us the entry point had to be the web application on port 80, and that the Squid proxy on port 3128 was a lead worth revisiting once a foothold on the web layer was found — it hinted that SSH might become reachable indirectly by routing traffic through that proxy.

Next step: With port 80 confirmed open, the target's website was opened in a browser to begin web enumeration.

5. Web Enumeration

With HTTP confirmed open, the website itself became the main target. This section covers viewing the site in a browser and inspecting its raw HTML source to understand how the login mechanism worked before attempting to exploit it.

Step 4 — Viewing the Target Website in a Browser

Command: Browsed to http://192.168.56.103

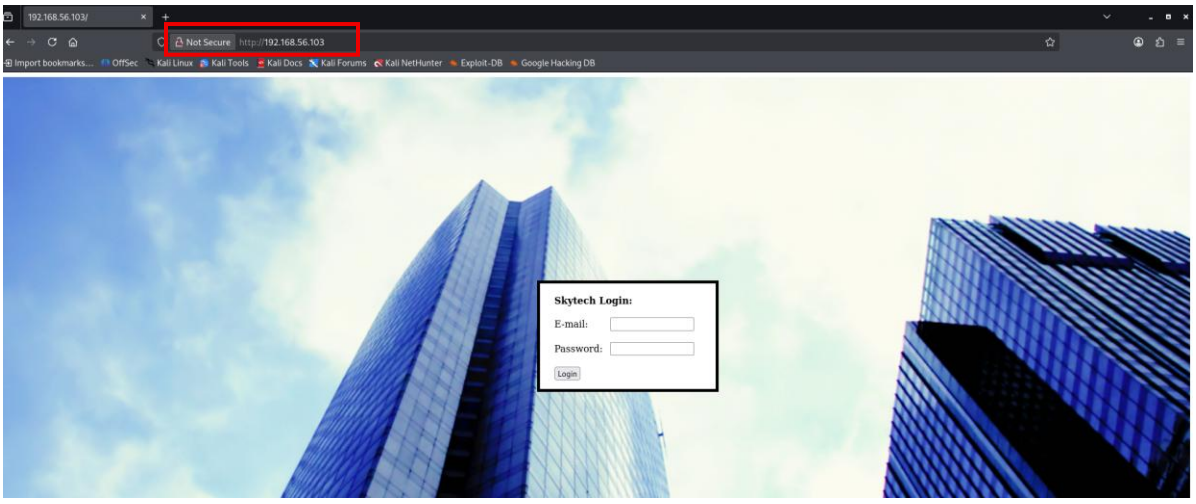


Figure 4: SkyTech Login Page

Description: The homepage displayed a corporate-style landing page for a fictional company, “SkyTech,” with a login box requesting an e-mail address and a password.

Result: A standard HTML login form was presented, with no other visible links or content on the page.

Interpretation: The only interactive element on the site is this login form, so the authentication mechanism itself became the natural attack surface.

Why this matters: Before trying to break the login, it was necessary to understand exactly how the form submitted data — the field names, the HTTP method, and the target script — which meant looking at the underlying HTML source rather than just the rendered page.

Next step: curl was used to fetch the raw HTML of the page and extract the form details.

Step 5 — Extracting the Login Form Details with curl

Command: curl -s http://192.168.56.103 | grep -E -i 'form|input|action|name='

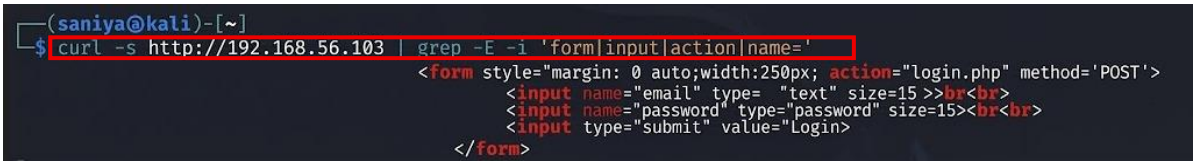


Figure 5: Raw HTML of the Login Form

Description: curl downloaded the page's raw HTML, and grep filtered the output down to only the lines relevant to the login form, rather than reading through the entire page source manually.

Result: The form's action attribute pointed to login.php using the POST method, with two input fields: email (a text field) and password (a password field), plus a submit button.

Interpretation: The credentials are POSTed to login.php as email and password. Because the field is literally named email rather than username, and because it is processed server-side (most likely by a SQL query against a database), it is a reasonable candidate for a SQL injection test.

Why this matters: Knowing the exact field names, the target script, and the HTTP method is what makes it possible to test the form deliberately, rather than guessing blindly at the login box.

Next step: The login form was tested directly in the browser using a classic SQL injection payload in the email field.

6. Exploitation — SQL Injection Authentication Bypass

Step 6 — Testing the Login Form for SQL Injection

Command : Submitted the email field as '*' with any value in the password field

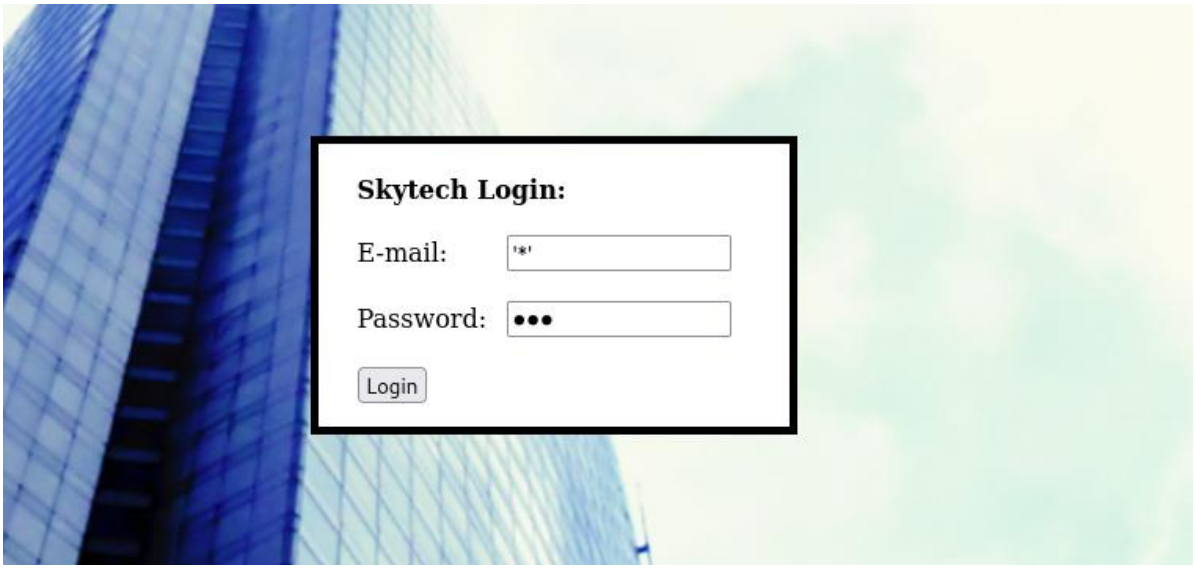


Figure 6: SQL Injection Payload Submitted in the Login Form

Description: A single quote followed by a wildcard ('*') was entered into the e-mail field. This is a classic probe for authentication bypass: if the back-end query concatenates user input directly into a SQL statement without sanitisation, breaking out of the intended string with a quote and injecting a wildcard can cause the query to match every row in the table instead of a single user.

Result: The form accepted the payload and submitted it to login.php without any visible input validation error.

Interpretation: The absence of any filtering or error on this kind of payload is a strong indicator that the login query is vulnerable to SQL injection, since a normal application would reject or escape a lone quote character.

Why this matters: If the query behind this form can be manipulated, authentication can potentially be bypassed entirely without needing to know a valid e-mail address or password.

Next step: The response returned by the server after submitting the payload was reviewed to see whether the bypass had succeeded.

Step 7 — Bypassing Authentication and Retrieving Leaked Credentials

Command: Reviewed the page returned after submitting the injected login request

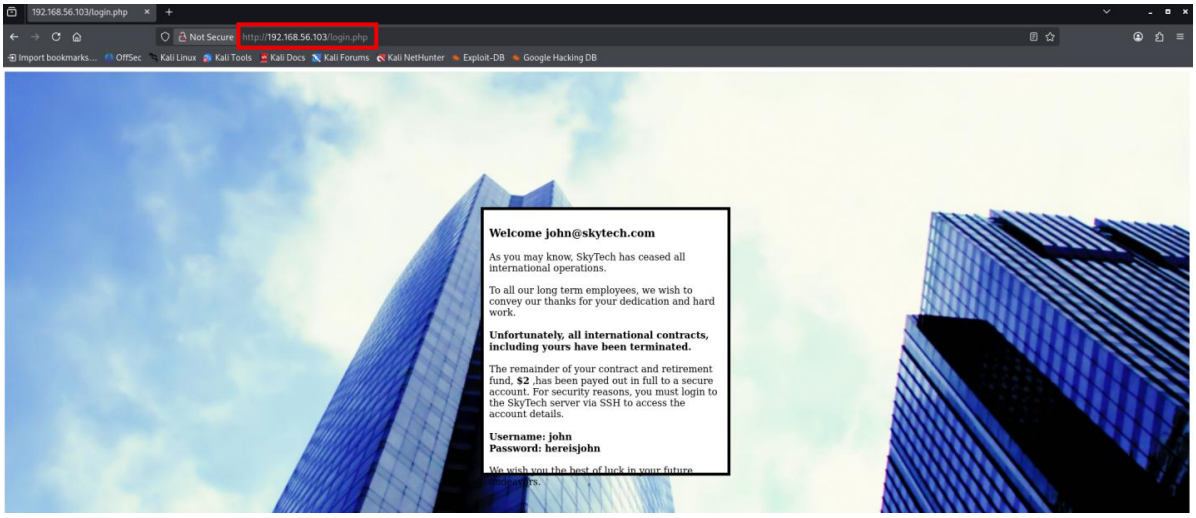


Figure 7: Authenticated Response — Leaked SSH Credentials

Description: After the injected request was submitted, the application redirected to login.php and rendered an authenticated welcome page instead of the login form, addressed to the first account the wildcard query matched.

Result: The page greeted "john@skytech.com" and displayed an internal memo about the company's closure. Buried in the memo were explicit SSH credentials for the SkyTech server: username **john** and password **hereisjohn**.

Interpretation: The SQL injection successfully bypassed authentication, confirming the vulnerability. More importantly, the authenticated content itself leaked a working set of operating-system-level credentials, directly bridging the gap from a web-layer bug to a login on the target host.

Why this matters: This is the pivotal finding of the assessment: a single injection flaw in a web form led straight to valid SSH credentials, which is exactly the kind of high-impact, low-effort exposure that penetration tests are designed to catch.

Next step: Recall that Nmap had shown port 22 (SSH) as filtered when scanned directly, while port 3128 (Squid, an HTTP proxy) was open. The credentials therefore needed to be used through that proxy rather than by connecting to port 22 directly.

7. Pivoting Through the Internal Proxy

Because SSH was filtered from the attacker's network but the Squid proxy on port 3128 was reachable, the logical next step was to route the SSH connection through that proxy so that, from the target's own perspective, the connection would originate locally rather than from an external filtered address.

Step 8 — Configuring Proxychains to Use the Squid Proxy

Command: sudo nano /etc/proxychains4.conf



Figure 8: Opening proxychains4.conf for Editing

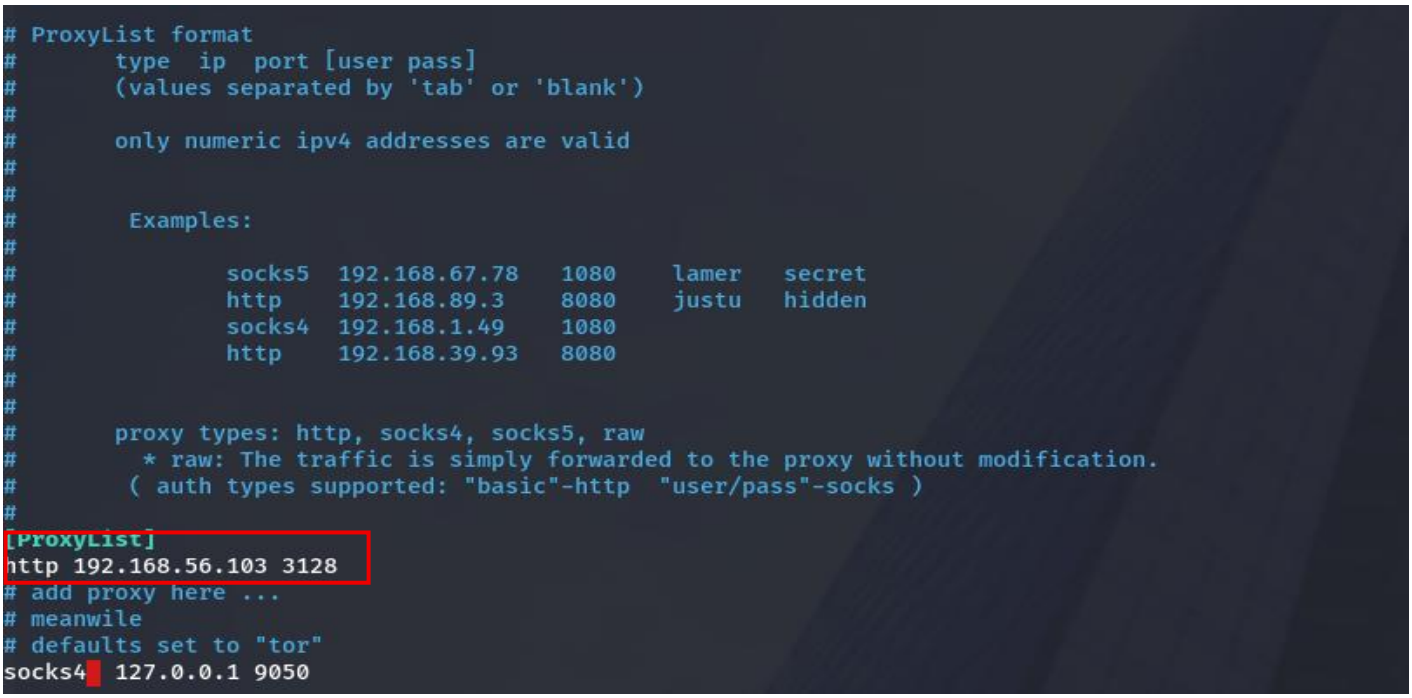


Figure 9: Proxy Entry Added to the ProxyList

Description: proxychains4 is a tool that forces the network traffic of any command it wraps through a chain of one or more proxies defined in its configuration file. The configuration file was opened and a new entry was added to the [ProxyList] section**: http 192.168.56.103 3128**, pointing at the Squid proxy discovered on the target.

Result: The proxy list now included the target's own Squid proxy as the routing point for any traffic sent through proxychains4.

Interpretation: Any subsequent command prefixed with proxychains4 would have its TCP connections routed through **192.168.56.103:3128** before reaching their final destination — effectively using the target's own proxy service to reach the target's own filtered SSH port.

Why this matters: This is the mechanism that turns an internal-only service (SSH, filtered from the outside) into something reachable, by "bouncing" the connection off a service that is exposed and permitted to reach it.

Next step: An SSH connection to the target was attempted using proxychains4 and the credentials found in the leaked memo.

Step 9 — Connecting Over SSH Through the Proxy

Command: proxychains4 ssh john@192.168.56.103

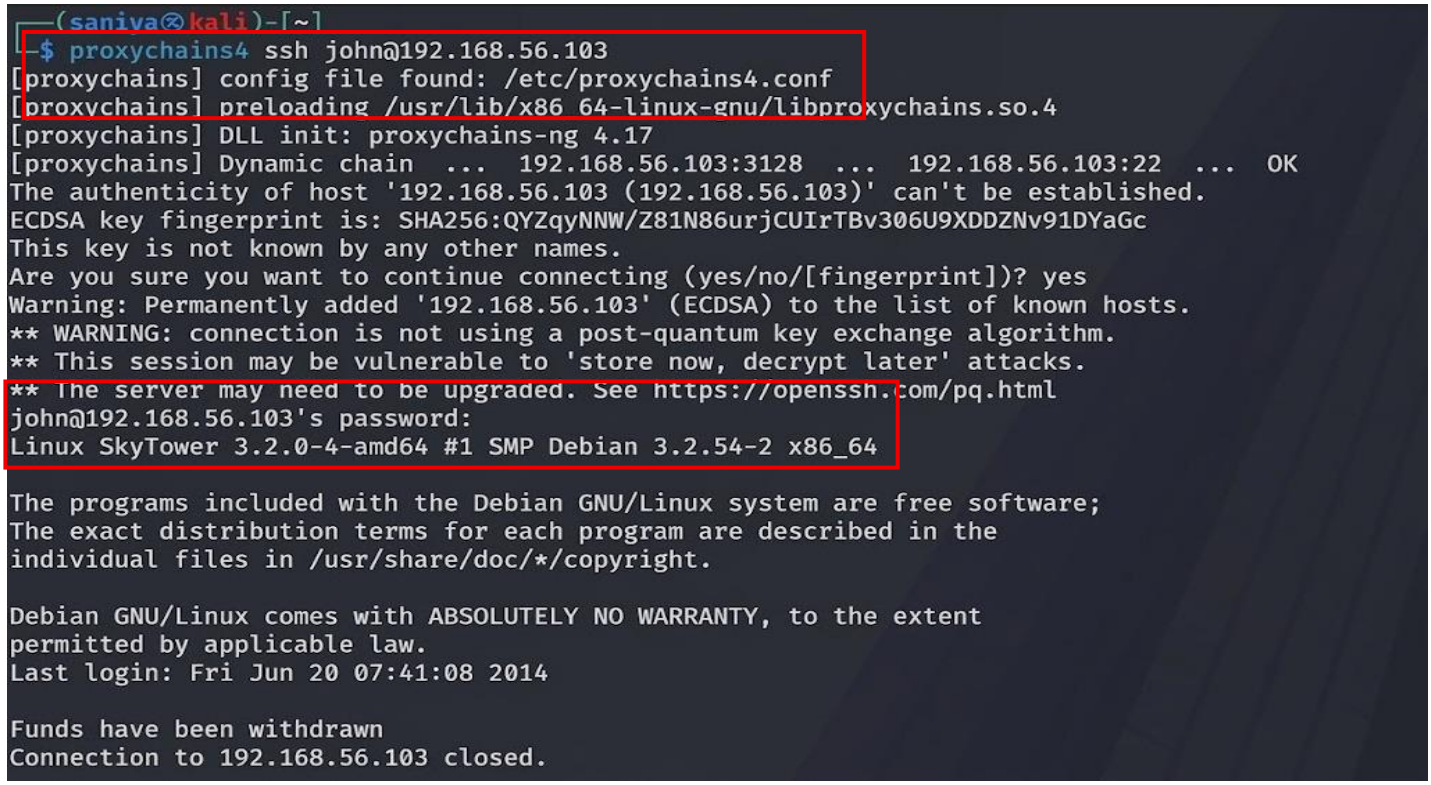


Figure 10: Initial SSH Connection Through Proxychains

Description: The ssh command was wrapped with proxychains4 so that the TCP connection to port 22 would be relayed through the Squid proxy on port 3128, as configured in the previous step. The credentials **john / hereisjohn**, found in the leaked memo, were supplied at the password prompt.

Result: Proxychains successfully chained the connection through **192.168.56.103:3128** to 192.168.56.103:22, and the password was accepted. The login banner displayed "**Linux SkyTower 3.2.0-4-amd64**" confirming the machine's hostname, followed immediately by the message "Funds have been withdrawn. Connection to 192.168.56.103 closed," after which the session terminated on its own.

Interpretation: The credentials were valid and the proxy pivot worked, but the account's login process appears to run a script or shell configuration that deliberately displays a message and then forcibly closes the connection before an interactive shell can be used.

Why this matters: Having valid credentials was not enough on its own — the account's shell environment was actively working against interactive access, meaning a way to bypass that automatic disconnect was required.

Next step: Instead of allowing SSH to start the account's default login shell, a specific replacement shell was requested directly on the SSH command line, bypassing the mechanism responsible for the automatic disconnect.

8. Shell Stabilization

Step 10 — Bypassing the Forced Restrictive Shell

Command: proxychains4 ssh -t john@192.168.56.103 "/bin/bash --noprofile --norc"

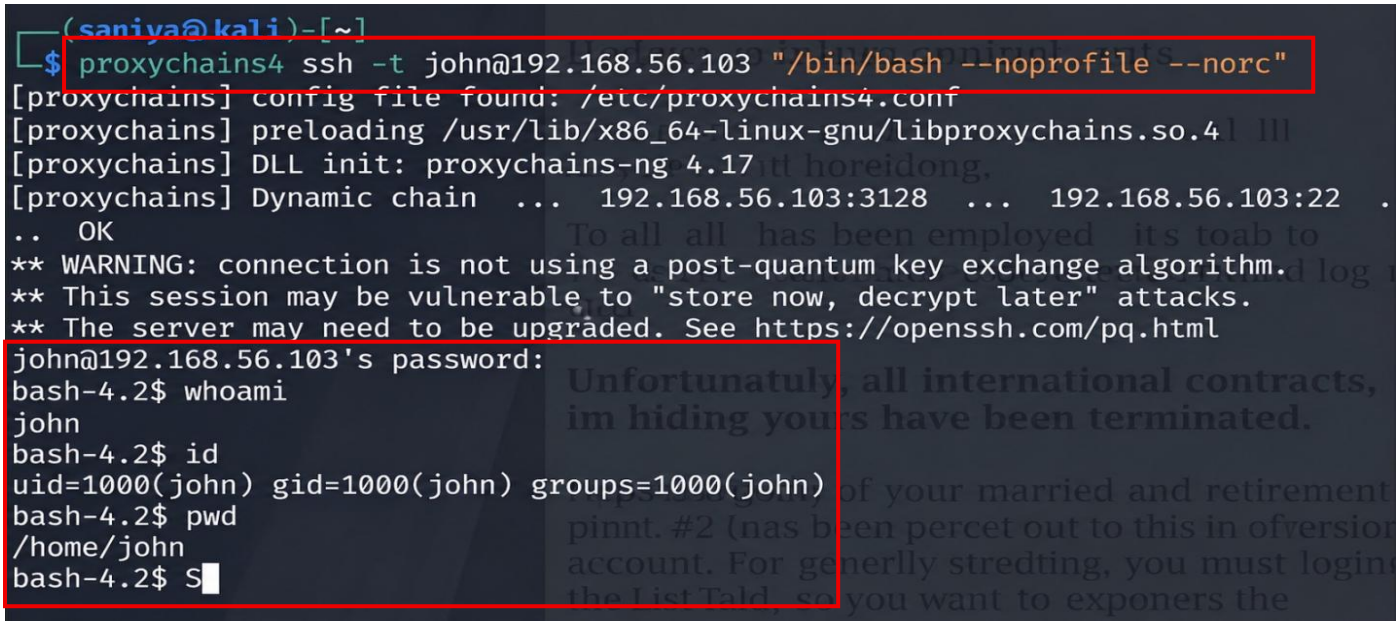


Figure 11: Bypassing the Auto-Disconnect by Requesting a Clean Bash Shell

Description: The -t flag forces SSH to allocate a pseudo-terminal, and specifying "/bin/bash --noprofile --norc" as the remote command tells the server to run that exact shell instead of john's configured default login shell. The --noprofile and --norc flags stop Bash from reading /etc/profile or the user's own .bashrc, which is where a startup script triggering the disconnect was suspected to live.

Result: The connection succeeded and stayed open. whoami confirmed the session was running as john, id showed uid=1000(john) gid=1000(john) groups=1000(john), and pwd confirmed the home directory /home/john.

Interpretation: Skipping the shell's normal startup files was exactly what was needed to keep the session alive, confirming that the disconnect behaviour was triggered from within one of those startup files rather than from the SSH server configuration itself.

Why this matters: This gave a stable, interactive foothold as the john user, which was necessary to continue any further local enumeration.

Next step: The user's home directory was inspected to find and neutralise the file responsible for the earlier disconnect.

Step 11 — Removing the Malicious .bashrc

Command: ls -la ; rm .bashrc

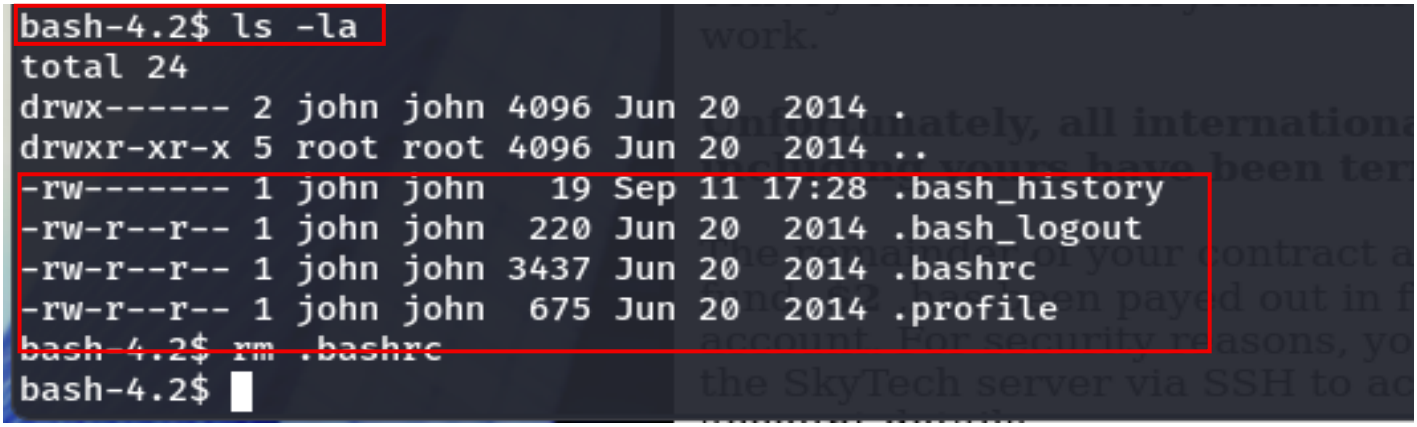


Figure 12: Home Directory Listing and Removal of .bashrc

Description: ls -la listed all files in john's home directory, including hidden dotfiles. Once .bashrc was identified as the only unusual startup-related file with a non-trivial size (3437 bytes, considerably larger than a stock file), it was deleted.

Result: The directory listing showed **.bash_history, .bash_logout, .bashrc, and .profile**. The .bashrc file was removed successfully with no error.

Interpretation: The size and content of .bashrc made it the most likely place for a startup routine that displays the withdrawal message and terminates the session, since that file is automatically sourced by Bash on every new interactive login.

Why this matters: Removing this file allows any future, unrestricted SSH login as john to reach a normal interactive shell without needing the --noprofile --norc workaround every time.

Next step: A fresh SSH connection was made, this time without specifying a replacement shell, to confirm that the disconnect behaviour was gone.

Step 12 — Confirming a Stable Shell

Command: proxychains4 ssh john@192.168.56.103

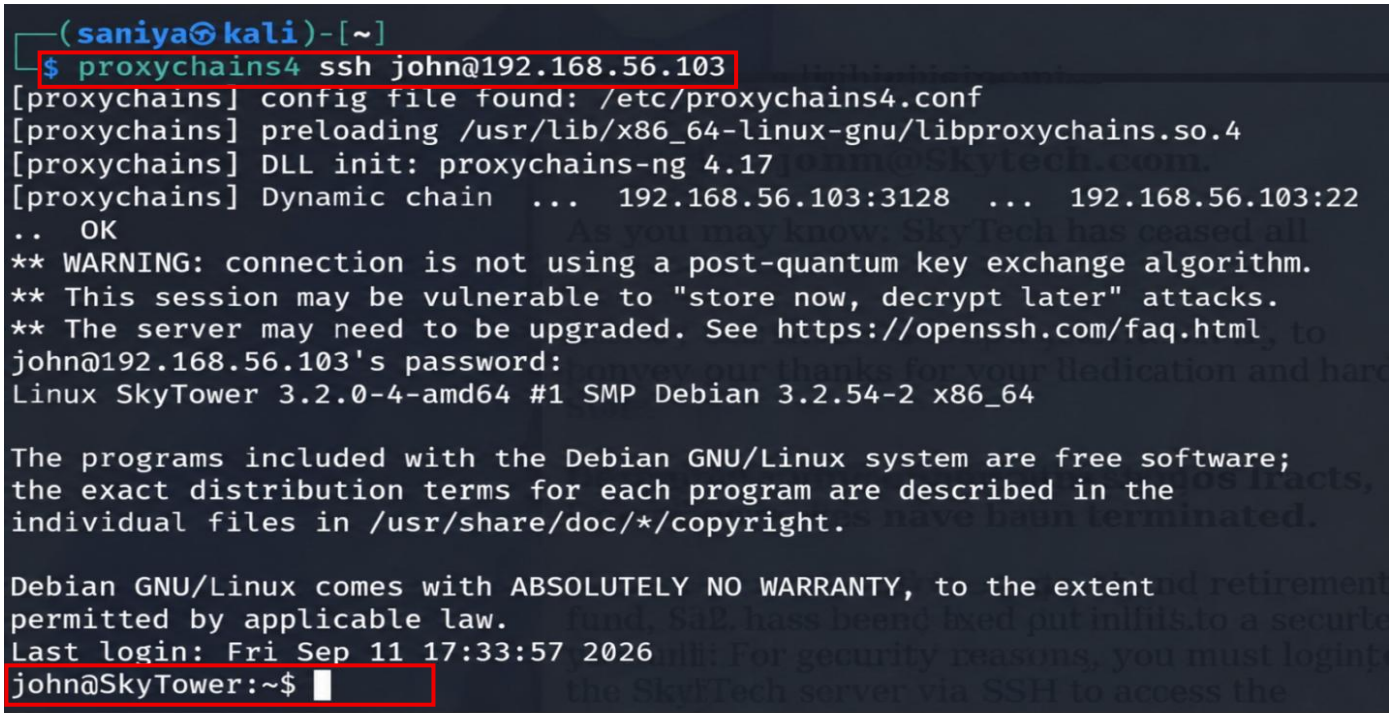


Figure 13: Normal SSH Login Now Reaching a Persistent Shell

Description: The same proxied SSH login used earlier was repeated, this time using the account's default shell rather than an explicit replacement.

Result: The login succeeded and, instead of immediately disconnecting, presented a normal, persistent shell prompt: **john@SkyTower:~$.**

Interpretation: This confirmed that .bashrc was indeed the source of the earlier automatic disconnect, and that the foothold on the box was now fully stable.

Why this matters: A stable, ordinary shell makes every subsequent enumeration step far simpler and less error-prone than working around a session that forcibly closes itself.

Next step: With a stable shell as john, local enumeration began to look for a path toward higher privileges.

9. Local Enumeration

Step 13 — Checking sudo Privileges and Network Connections

Command: sudo -l ; netstat -antp

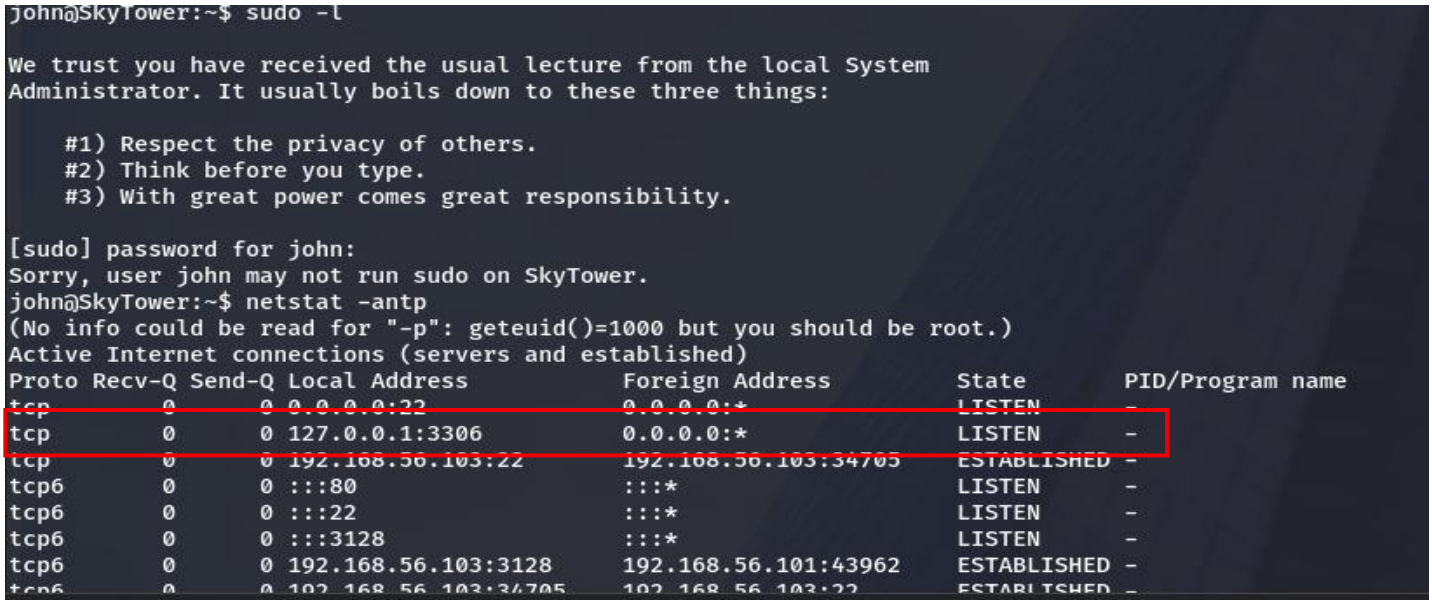


Figure 14: sudo Privilege Check and Active Network Connections

Description: sudo -l lists any commands the current user is permitted to run as another user. netstat -antp lists active and listening TCP connections along with the associated process, when permissions allow it.

SkyTower,

Result: sudo -l returned “Sorry, user john may not run sudo on ” ruling out a direct sudo-based escalation for this account. netstat showed, among other listening ports, **127.0.0.1:3306** — a MySQL server bound only to localhost — alongside the already-known services on ports 22, 80, and 3128.

Interpretation: John cannot escalate through sudo, so another route was needed. The MySQL service bound to localhost was a new lead: since it was not reachable from the network at all, any credentials for it would have to be found locally on the box itself, most likely inside the web application's source code.

Why this matters: This redirected the investigation toward the web application files under john's reach, specifically looking for the database credentials that login.php would need to connect to MySQL.

Next step: The PHP source of the login page was read directly from disk.

Step 14 — Reviewing Web Application Source for Database Credentials

Command: cat /var/www/login.php

```
john@SkyTower:~$ cat /var/www/login.php
<?php

$db = new mysqli('localhost', 'root', 'root', 'SkyTech');

if($db->connect_errno > 0){
    die('Unable to connect to database [' . $db->connect_error . ']');
```

Figure 15: MySQL Credentials Hardcoded in login.php

Description: The source code of the vulnerable login page was read directly, since it was readable by john and was already known to build a database connection for the login form.

Result: The file showed a call to **mysqli('localhost', 'root', 'root', 'SkyTech')** — the database was accessed using the username root and the password root, against a database named SkyTech.

Interpretation: The web application connects to MySQL as the root database user, using a trivially weak, default-looking password. Because the MySQL service is bound to localhost only, these credentials are directly usable now that there is a local shell on the box.

Why this matters: This gave a second, independent route to sensitive data: instead of continuing to exploit the web form over HTTP, the same backend database could now be queried directly from the local shell, likely exposing every account's stored credentials at once.

Next step: The mysql client was used locally to connect with the discovered root credentials and inspect the SkyTech database.

10. Credential Discovery — MySQL Database

Step 15 — Logging into MySQL with the Discovered Credentials

Command: mysql -uroot -proot

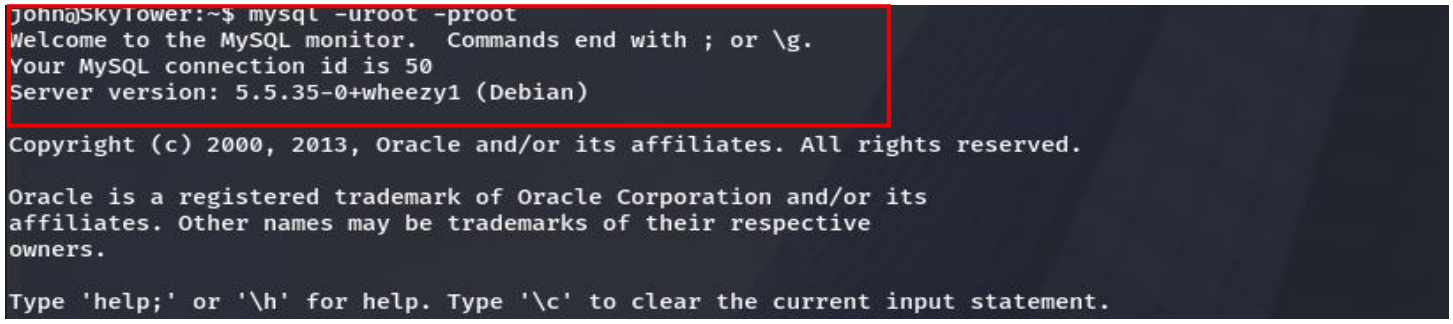


Figure 16: Successful Local MySQL Login as root

Description: The mysql command-line client was used to connect to the local MySQL server with the username and password recovered from login.php.

Result: The login succeeded immediately, dropping into the MySQL monitor prompt.

Interpretation: The credentials found in the source code were correct and gave full administrative access to the MySQL server.

Why this matters: Full database access means any table in any database on this server can now be read, including whatever the SkyTech application stores about its users.

Next step: The SkyTech database was selected so its tables could be examined.

Step 16 — Selecting the SkyTech Database

Command: use SkyTech;

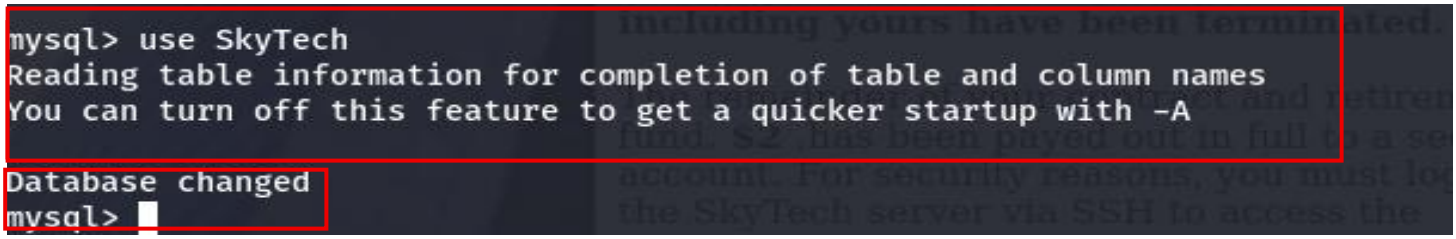


Figure 17: Switching to the SkyTech Database

Description: The USE statement switches the active database context for the current MySQL session to the database named SkyTech, the same one referenced in login.php.

Result: MySQL confirmed "Database changed," making SkyTech the active database for subsequent queries.

Interpretation: This is the same database the login form queries, so any credential table used for authenticating website users should be located here.

Why this matters: Working in the correct database context was necessary before listing or reading any of its tables.

Next step: The tables inside SkyTech were listed, and the relevant one was dumped.

Step 17 — Dumping the Login Table

Command: show tables; select * from login;

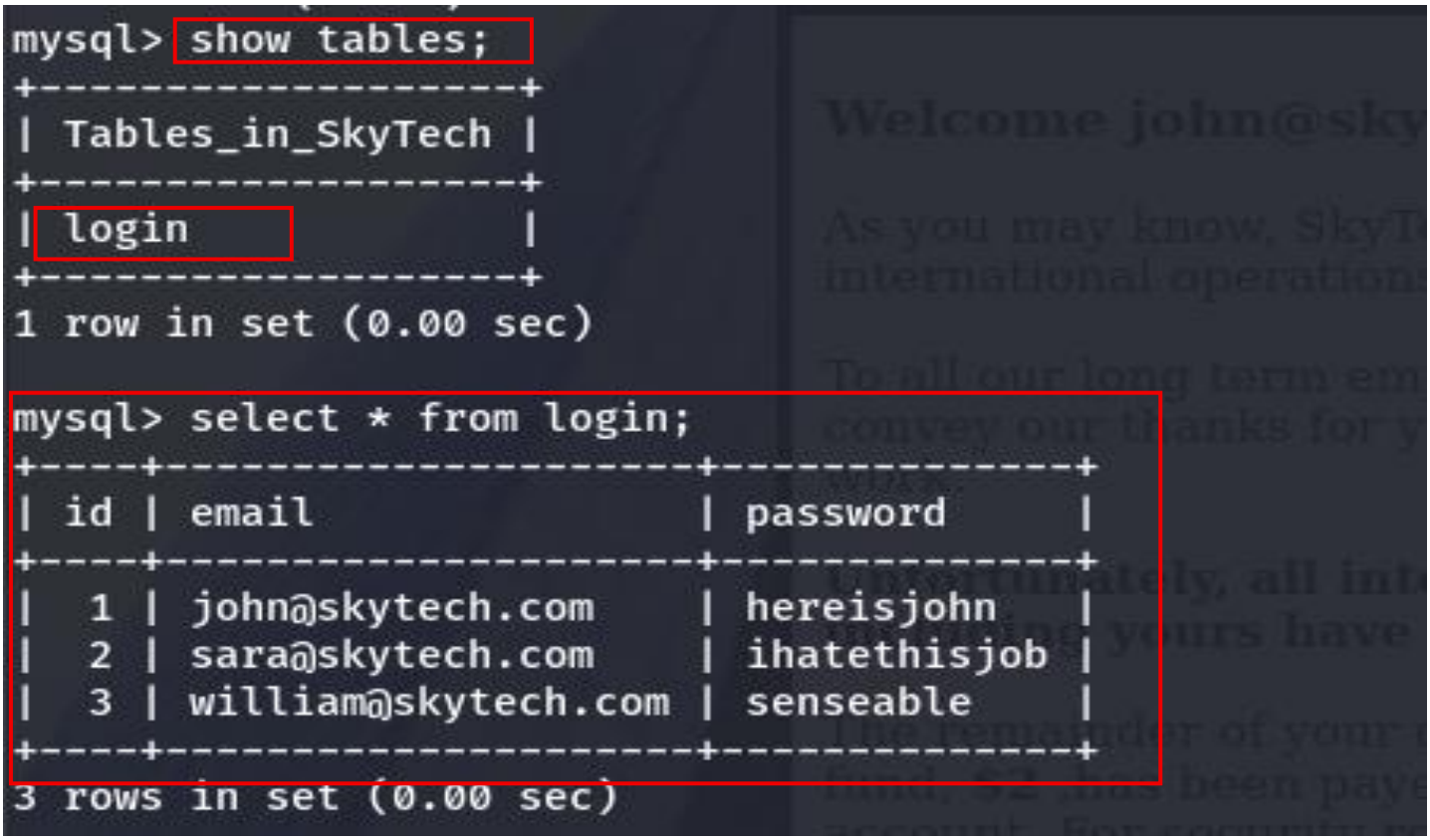


Figure 18: Contents of the login Table

Description: show tables listed every table inside the SkyTech database, and select * from login dumped the full contents of the one relevant table found.

Result: The database contained a single table, login, with three rows: **john@skytech.com / hereisjohn**, **sara@skytech.com / ihatethisjob**, and william**@skytech.com / senseable**.

Interpretation: This confirmed the credential john / hereisjohn already in use, and revealed two additional sets of plaintext credentials for the accounts sara and william, stored in the same table that the vulnerable login form queries.

Why this matters: These additional credentials are strong candidates for other local operating-system accounts on the box, since password reuse between a web application and system accounts is a common misconfiguration on machines like this one.

Next step: The newly discovered credentials were tried against the local operating-system accounts to look for a path toward privilege escalation.

11. Lateral Movement — Pivoting to the sara Account

Step 18 — Switching to sara Using the Discovered Credentials

Command: ssh sara@localhost -t /bin/sh

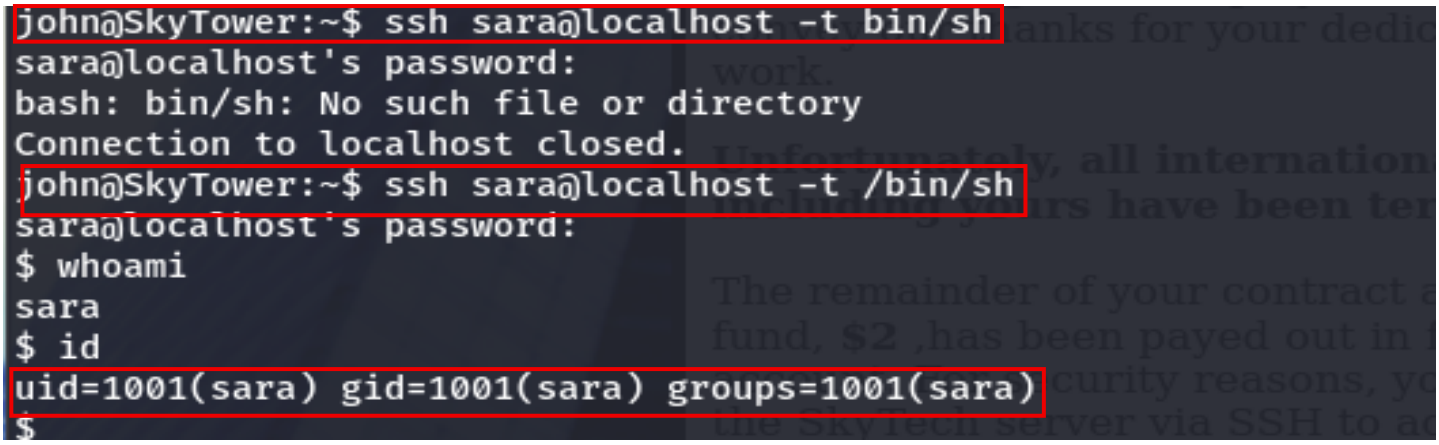


Figure 19: Successful Login as sara via Local SSH

Description: Rather than su, an SSH connection back to the machine itself (localhost) was used to authenticate as sara with the password ihatethisjob taken from the database dump, forcing the interactive shell /bin/sh so as to avoid any similar restrictive-shell tricks tied to sara's own account configuration.

Result: The first attempt with a mistyped path failed with “bin/sh: No such file or directory,” and the connection closed. A corrected attempt using the absolute path /bin/sh succeeded: **whoami returned sara**, and id confirmed **uid=1001(sara) gid=1001(sara) groups=1001(sara).**

Interpretation: The password recovered from the MySQL database was indeed reused as sara's operating-system login password, confirming password reuse across the web application and the local system accounts.

Why this matters: This is a fresh set of privileges to enumerate; a different account may well have different, more permissive sudo rights than john did.

Next step: sara's sudo privileges were checked next.

12. Privilege Escalation — Sudo Wildcard Exploitation

Step 19 — Enumerating sara's Sudo Rights

Command: sudo -l

```
$ sudo -l
Matching Defaults entries for sara on this host:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bi
n

User sara may run the following commands on this host:
    (root) NOPASSWD: /bin/cat /accounts/*, (root) /bin/ls /accounts/*
$
```

Figure 20: sara's Permitted Sudo Commands

Description: sudo -l, run as sara, lists exactly which commands this account is permitted to execute as another user (in this case root) without needing further authorisation beyond the account's own password.

Result: The output showed: **(root) NOPASSWD: /bin/cat /accounts/*, (root) /bin/ls /accounts/***. This means sara can run cat and ls as root, but only against paths matching the pattern /accounts/*.

Interpretation: The wildcard in the sudoers rule is matched by the shell before sudo ever sees the expanded argument, and because /accounts/* is a glob rather than a fixed filename, an argument like /accounts/../../../root/anything is still textually inside /accounts/ from sudo's point of view, even though it resolves to a completely different directory once the shell interprets the path traversal.

Why this matters: This is a classic sudoers wildcard misconfiguration. Because cat and ls can be pointed at any path that merely starts with the literal string /accounts/, path traversal sequences can be used to read or list files far outside that directory — including root's own home directory — all with root's effective permissions.

Next step: The wildcard was exploited using ../ path traversal to reach and read root's home directory.

Step 20 — Exploiting the Wildcard to Read Root's Files

Command: sudo ls /accounts/../../../root ; sudo cat /accounts/../../../root/flag.txt

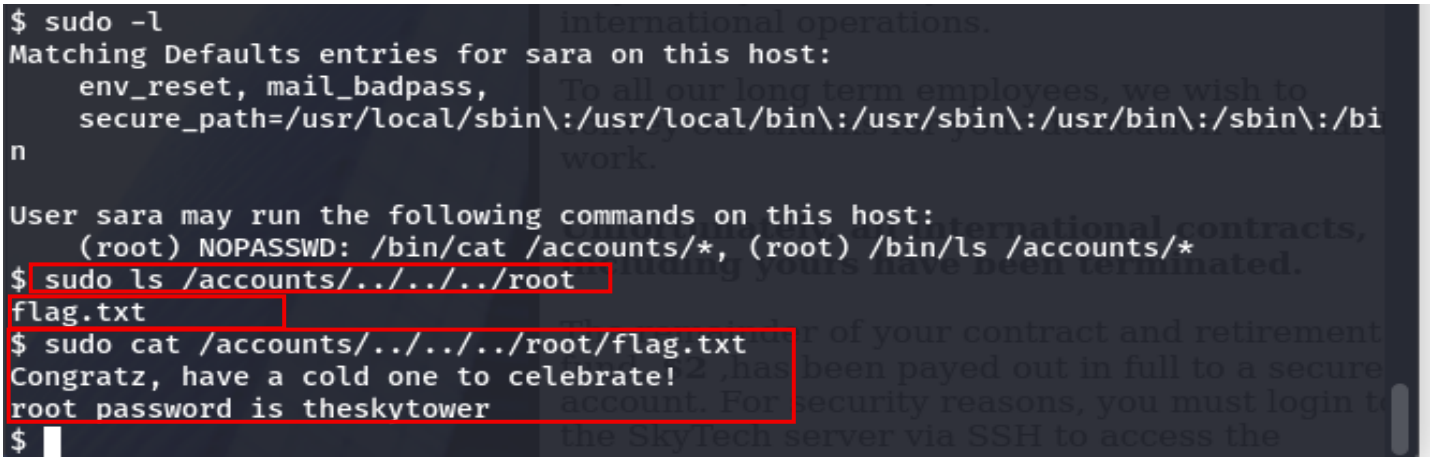


Figure 21: Root's Directory Listed and flag.txt Read via the Sudo Wildcard

Description: The permitted path /accounts/ was suffixed with ../../../root, a sequence of parent-directory references that, once resolved, points at /root instead. Because the argument still begins with the exact text /accounts/, it still satisfies the sudoers wildcard rule, so both the ls and cat commands ran as root against the real /root directory.

Result: sudo ls /accounts/../../../root listed a single file, **flag.txt. sudo cat /accounts/../../../root/flag.txt** then printed its contents: "**Congratz, have a cold one to celebrate! root password is theskytower**."

Interpretation: The sudoers wildcard misconfiguration allowed full read access to root's home directory, including a file that discloses the actual root account password for the machine, marking the conclusion of the challenge.

Why this matters: Root's own password is now known, meaning the machine is fully compromised: an authenticated root login (for example, su - or a direct SSH login as root) is now possible using this password, confirming complete privilege escalation from an unauthenticated web visitor to root.

13. Conclusion

The SkyTower: 1 machine was fully compromised, from initial reconnaissance to root-level access, by following the clues the machine itself provided at each stage. The attack chain relied on four main weaknesses chained together: a SQL injection flaw in the login form that both bypassed authentication and leaked SSH credentials in the resulting page content, an internal Squid proxy that could be used to reach an otherwise filtered SSH port, a restrictive .bashrc configured to forcibly terminate interactive sessions, hardcoded MySQL root credentials left inside the application's PHP source, password reuse between the web application's database and local operating-system accounts, and finally a wildcard sudoers misconfiguration that allowed path traversal into root's home directory.

Each stage of the report followed the same pattern: discover a clue, understand what it meant, test a theory based on it, and verify the result before moving to the next step. The final root password, theskytower, was successfully retrieved from /root/flag.txt, confirming complete compromise of the target machine.